BigPrint TV

Devpost submission — Build, Ship, Shape: Amazon Developer Hackathon, Fire TV track (Vega OS). Paste-ready; identical to SUBMISSION.md §4.

The letter you can't read anymore, in very large print on the TV you already own. Every number checked. Read aloud when you ask.

About line: Built in Pakistan, by someone who needed it, for every living room with a Fire TV. First market: the United States, where Fire TV and Vega OS live.

Inspiration

A few months ago I damaged my eyes looking at the sun. Solar retinopathy. The scar is one tiny spot, right in the centre of my vision, which happens to be exactly where reading lives. My phone turned into a smudge. The post on the table turned into shapes. A bill, a letter from the clinic, a note from a friend: I could hold them, I just couldn't read them. I kept wishing for something very simple, a big reader for text.

Then one evening I looked up, and it was already in the room. The biggest screen in the house, two metres away, with a remote I could use with my eyes closed. Millions of people live with sight loss, most of them older, and their mail never stops arriving. Their television is already there too. So I built the thing I was wishing for.

What it does

The TV shows a QR code. A relative or caregiver scans it with any phone and gets a page with one button: photograph the letter. That's the phone's whole job, it is only a camera.

About ten seconds later the letter is on the TV in large print, three ways: the whole letter in plain English, just the main points, or the exact words as printed. The figures nobody can afford to get wrong, the amount due, the pay-by date, the account number, are pinned at the top and never scroll away. Press Listen and the whole letter is read in a calm, natural voice. Press Next photo for the next page. Text goes up to twice the size, and there are three looks, dark, light, and yellow on black, chosen on first run by reading a real sentence in each, because eyes disagree about what is easiest, even with the same condition. Letters stay on the TV for a day, so one closed by mistake is two presses back, and then everything is deleted. It is someone's private mail.

How we built it

React Native 0.83 on Vega OS for the TV app. Behind it, two AWS Lambda functions and one S3 bucket, nothing running between letters. Amazon Textract reads the photo with layout and tables, so a bill's rows stay rows. Amazon Nova Lite classifies the page. Textract's expense model pulls the money figures, and Nova Pro looks at the photo independently with one power only: it can veto a figure it disagrees with, never add one. Nova Pro then rewrites the page into short spoken sentences, and two guards run last: a currency the page does not print is never spoken, and any sentence containing a number that is not on the page is dropped. Amazon Polly makes the voice, only when someone presses Listen, so a letter nobody listens to costs nothing to voice. The phone page converts iPhone HEIC photos to JPEG in the browser.

Challenges we ran into

The honest list is long, and ten entries of it are written up in our friction log. The ones that shaped the product: our first OCR pass turned a receipt's table into "the amount for diesel is 8", right digits, wrong row, which is where the layout-aware OCR and the numeric guard were born. Textract's expense model confidently found a SUBTOTAL and TAX on a university certificate, so now the pipeline classifies first. A Lambda Function URL configured exactly as documented refused everyone with a 403; an API Gateway in front fixed it without a code change. Read-aloud was silent until the manifest declared audio services the docs don't mention. And the build broke because our folder name had a space in it.

Accomplishments that we're proud of

The thing we are proudest of is what the app refuses to do. Numbers never come from the language model. A disputed figure is hidden, not shown. On our sample photographs, every key fact the TV displayed was correct, 17 of 17, and when a page is blurry the TV says "Check: some words were hard to read" instead of guessing. For a reader who cannot easily check the paper, a confidently wrong number is worse than no number, so we built the whole pipeline around that sentence.

We are also proud of the accessibility work: every colour pair tested at WCAG 7:1 in the test suite, and the non-obvious Fire TV fact that the screen reader follows input focus, so the pinned key facts had to be linked to the first focusable line or a blind user would never hear the amount due, which is the entire point of the app. Scam post targets exactly this audience, so a letter is treated as data, never as instructions, and the public endpoint cannot call any AI service at all. 157 app tests and 37 backend tests.

What we learned

That reading is the product and the voice is a gift, so the text must arrive first and the voice should be made on request. That a language model must never be the source of a number. That older eyes disagree about polarity even within one eye condition, so the viewer should choose by reading, not by name. And that a friction log kept from day one is worth more than memory.

What's next

A run on a physical Fire TV with VoiceView, the screen reader, which we have not done yet and say so plainly. Then distribution through the people who actually reach this audience: adult children, caregivers, and sight-loss organizations, such as the American Foundation for the Blind and Prevent Blindness in the US, and the RNIB and the Macular Society in the UK. A per-TV allow-list so only a paired phone can send, CloudFront in front of the API, and a batch mode so a caregiver can photograph a whole pile of post at once. A letter read on screen costs about two to three cents, which makes a free or charity-funded service realistic.

Built with

react-native, vega-os, fire-tv, aws-lambda, amazon-api-gateway, amazon-s3, amazon-textract, amazon-bedrock, amazon-nova, amazon-polly, typescript, python